

Introduction

In this lab, I worked with a Neural Style Transfer (NST) implementation based on a Jupyter Notebook that was provided to me. The original notebook ran only a single experiment at a time, so I modified it to carry out multiple experiments automatically. The goal was to apply style transfer on different image pairs and compare how different hyperparameters affect the output. In the second part, which I will do later, I will also compare how different CNN layers influence the results.

Experimental Setup

I used PyTorch and a pretrained VGG network for feature extraction like in the lecture notebook. Instead of running the style transfer manually for each single image, I rewrote the notebook so that it automatically loads all content images in the images directory and all style images in the styles directory and processes every possible combination. This let me to run dozens of experiments programmatically.

The style transfer function was also modified so it returns the final content and style losses, which I stored for result comparison later.

Dataset

I didn't use a specific named dataset. Instead, I placed my chosen content and style images inside two folders:

- images/ – content images
- styles/ – style images

I modified the code so all images in these directories are loaded using a simple loop. This turned the entire NST process into a batch procedure so I didn't need to run each trial manually.

Configuration

The main thing I tested in this first stage was different hyperparameter settings, while keeping the CNN layers at their default configuration. The hyperparameters I varied were:

- Number of optimization steps
- Style loss weight
- Content loss weight

The exact hyperparameter combinations I tested were:

Experiment	Steps	Style Weight	Content Weight
1	100	10k	1
2	200	5k	1
3	100	100k	1
4	150	3k	10
5	80	15k	1

Each setting was applied to every content–style pair.

For the layer experiment

Name	Content Layers	Style Layers
Default	conv_4	conv_1, conv_2, conv_3, conv_4, conv_5
Experiment 1	conv_4	conv_1, conv_2
Experiment 2	conv_3	conv_4, conv_5
Experiment 3	conv_4	conv_3, conv_4, conv_5

Code Modifications

Loading All Images at Once:

I first changed the code so that instead of loading only one content and style image, the program loads all of them:

```
1 image_paths = glob.glob("images/*")
2 style_paths = glob.glob("styles/*")
3 content_images = [image_loader(image_path) for image_path in image_paths]
4 style_images = [image_loader(style_path) for style_path in style_paths]
```

Running Every Combination:

Originally, the notebook only processed one experiment at a time. I replaced that with nested loops that combine:

- every hyperparameter configuration
- every content image
- every style image

Returning Final Loss Values:

I modified `run_style_transfer()` so that the final style and content losses are recorded and returned. I then appended these results to a list so I could analyze them later.

Triplet Display Function

I wrote a new helper function that shows:

- the content image
- the style image
- the final generated image

side by side in one figure.

```
1 def imshow_triplet(content_tensor, style_tensor, output_tensor,
2                     titles=("Content", "Style", "Output")):
3     def to_img(tensor):
4         image = tensor.cpu().clone().squeeze(0)
5         return unloader(image)
6     content_img = to_img(content_tensor)
7     style_img = to_img(style_tensor)
8     output_img = to_img(output_tensor)
9     plt.figure(figsize=(15, 5))
10    images = [content_img, style_img, output_img]
11    for i, (img, title) in enumerate(zip(images, titles)):
12        plt.subplot(1, 3, i + 1)
13        plt.imshow(img)
14        plt.title(title)
15        plt.axis("off")
16    plt.tight_layout()
17    plt.show()
```

Full Loop

In my loop, I apply all the styles to images I have with different combinations of hyperparameters and different sets of layers, at the same time, I save up all the results into a list of dictionary where I store experiment name, content and style layers, hyperparameters, style and content loss, index of the image and the style, combine score which is style and content loss summation, and finally the output image. Since the loop is pretty long because of all these savings I am not going to put the image of it, however at the end of it the loop, I pick the top 2 images that had the best loss.

Result Analysis

To isolate the effect of CNN layer choice on neural style transfer, I kept the hyperparameters fixed (100 steps, style weight = 10k, content weight = 1) and changed only which VGG layers contributed to the content and style losses. The results showed clear visual differences, and the behavior aligns both with theory and with the qualitative results I observed.

Default Configuration

- **Content:** conv_4
- **Style:** conv_1–conv_5

The default configuration served as a baseline and produced two very different outcomes:

1. Successful Result

One output gave a balanced stylization:

- the content image remained recognizable,
- colors and textures of the style image were integrated subtly,
- the output looked clean and natural.

This matches the expectation that combining all convolutional layers captures both low-level textures and high-level structure.



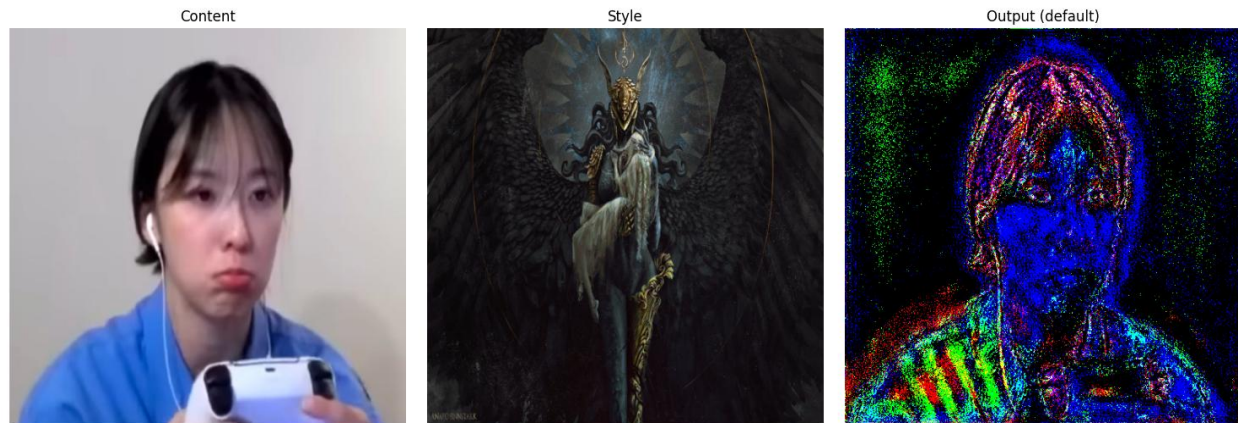
2. Failed Result (Black/Dark Output)

The second example from the same configuration produced extremely high loss and the image appeared black with random green and blue patches.

Although the style characteristics were not present, the content outline was still somewhat visible. This type of failure can occur when:

- the optimization diverges,
- gradients explode,
- the learning process collapses into a bad local minimum, especially in multi-layer setups where many competing constraints are applied at once.

This shows that while the default configuration is capable of strong results, it is also the most complex to optimize and can lead to instability if the optimization steps are not sufficient.



Experiment 1 – Only Early Style Layers

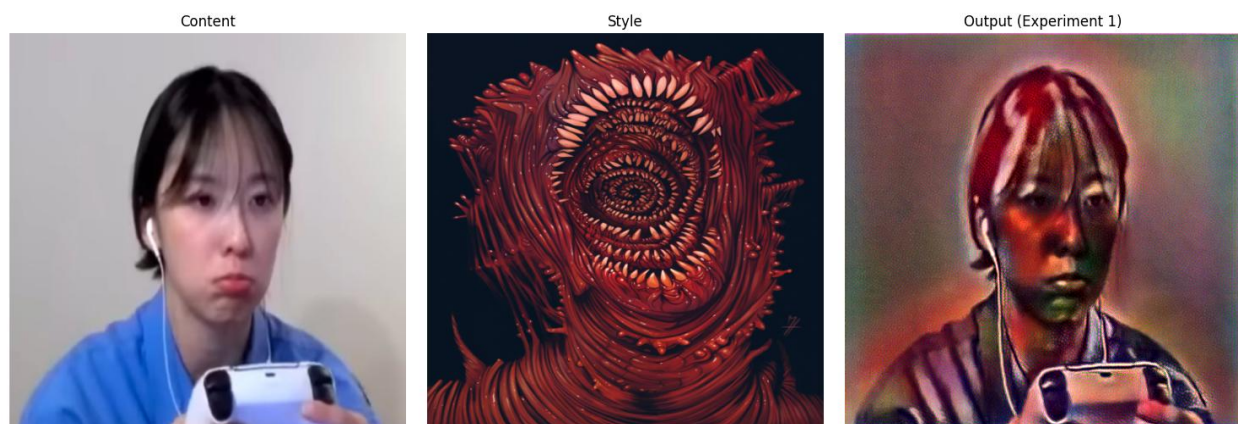
- **Content:** conv_4
- **Style:** conv_1 and conv_2

Here I obtained one result that worked reasonably well:

- The content image remained clean and recognizable.
- Noise was somewhat low, and the structure was preserved.
- However, the stylization was very weak:
instead of transferring brush strokes, patterns, or artistic structure, the effect resembled a color wash or color grading filter.

This observation is consistent with theory:

early CNN layers mainly encode edges, brightness, and small texture details. They do not capture high-level patterns or shapes, so the algorithm changed colors but did not impose the visual “character” of the style image.



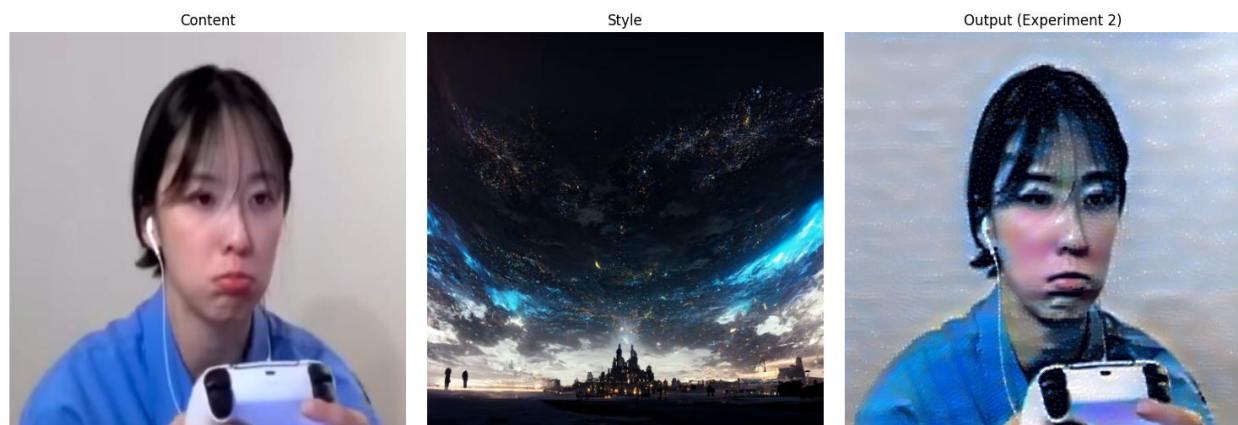
Experiment 2 – Only Deep Style Layers

- **Content:** conv_3
- **Style:** conv_4 and conv_5

Experiment 2 behaved somewhat like Experiment 1 in terms of visual subtlety, but the results were noticeably better:

- The content structure remained very clean and sharp.
- The stylization was visible but not overwhelming.
- The output had more coherence and aesthetic refinement compared to Experiment 1.

Using only deep layers means the model focuses on high-level structure and broad spatial patterns, rather than textures or local noise. In this case, that produced a controlled, gentle stylization without distortion. This confirms that deep layers influence the global styling while avoiding random pixel-level artifacts.



Experiment 3 – Mid + Deep Style Layers

- **Content:** conv_4
- **Style:** conv_3, conv_4, conv_5

This configuration gave the strongest and most successful stylization among the experiments. Using the same style image as Experiment 1 produced a dramatically more coherent result:

- Not only was the content image still recognizable,
- but the style integration was much richer:
 - the colors transferred clearly,

- stylistic shapes and forms were incorporated,
- the image appeared “morphed” with the style in a visually pleasing way.

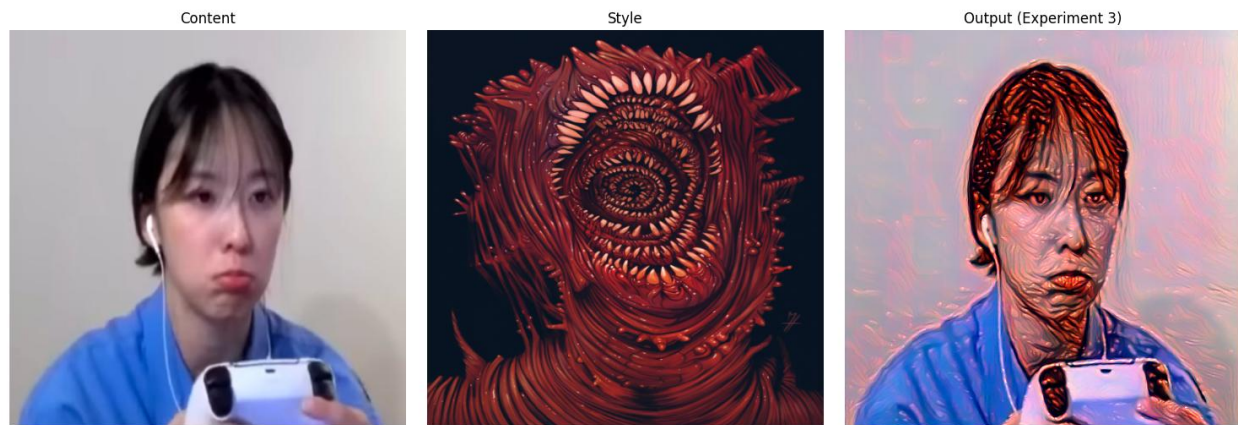
Compared to Experiment 1, which transferred mainly color but no structural identity, Experiment 3 captured both the texture and artistic geometry of the style image.

This matches theory:

mid and deep layers together encode both:

- medium-scale textures (conv_3), and
- large-scale style representation (conv_4–5).

This makes the stylization more expressive without destroying the scene structure.



Conclusion

The results confirmed the hierarchical nature of convolutional neural networks:

- Early layers (conv_1–conv_2) capture low-level information, such as color tone and fine textures. These experiments produced images that preserved the content clearly but showed weak stylization, resembling a color filter or grading rather than full artistic transfer.
- Deep layers (conv_4–conv_5) represent higher-level patterns and spatial structure. Using these layers generated more coherent stylization while keeping the content recognizable, though the effect was still somewhat subtle.
- Combining mid and deep layers (conv_3–conv_5) produced the strongest and most visually satisfying transformation: colors, shapes, and patterns from the style image were transferred while still maintaining the main structure of the content image.

These configurations delivered the best balance of texture, structure, and artistic expression.

- Using all layers (default configuration) has the potential to produce strong stylization but also increases optimization complexity. This was evident in the mixed results: some outputs were successful and stylistically rich, while others diverged and produced distorted or nearly black images due to unstable optimization.

Overall, the experiments demonstrated that layer selection is a critical factor in neural style transfer, sometimes even more influential than hyperparameters, once a stable configuration is found. The chosen baseline hyperparameters provided reliable convergence, allowing the differences between layer choices to be observed clearly.